

What I've built so far

The first stage is done: the MVP is built and it's looking really good. Right now it's being tested against some documents from last quarter, and I've already run it through an initial pilot, which came back solid.

WHAT IT CAN DO TODAY

- Reads both PDF documents and web pages.
- Handles long documents by splitting them into sections without losing the thread between them.
- Takes page snapshots and reads tables, images, and basic charts, not just plain text. Many allocation views live in dial graphics and tables rather than sentences, so this matters.
- Was given a "financial analyst brain": worked examples and house conventions built from the sample documents in your earlier email, so it reasons like an analyst rather than a generic AI.
- Produces one output spreadsheet per run in the agreed format: firm, date, source, sub-asset class, the call, and the exact supporting text with a page reference for every call.

HOW I KEEP IT TRUSTWORTHY

- Every call must quote the exact source text and page that supports it. Calls that can't be verified against the source are rejected rather than kept.
- A second, independent AI reviewer re-reads every call against the original source before it is accepted. Think of it as a built-in checker.
- Dial and chart graphics are verified by that independent reviewer looking at the page image itself, not just the page text, so allocation views published as dials are read reliably.
- Confidence is not the AI grading itself. It comes from a fixed, transparent set of rules (was the quote verified? did the checker agree? how direct was the language?), with the AI only supplying the raw ingredients. Anything low-confidence is automatically flagged for human review.
- Where the evidence is genuinely weak or unclear, the system says "uncertain" instead of guessing.
- Every run also produces a separate failures file alongside the results: calls the system attempted but rejected, for example because the supporting quote could not be verified, or the checker disagreed, each with the reason. This keeps rejected calls reviewable on their own, and keeps them distinct from "uncertain" calls. "Uncertain" means the document itself was ambiguous; a "failure" means the system found something but couldn't stand behind it.
- Runs currently use the Claude and Codex subscriptions you purchased.

STATUS AND NEXT STEP

- The pilot run on the first sample set is complete, and the results are attached. In the pilot run, the system found the large majority of the calls my own review identified, and where both made a call they agreed on direction over 90% of the time.
- A second test, another seven sources from last quarter, is now complete, and those results are attached as well, along with a one-page comparison against our own reference calls.
- For now the analysis is powered by the Claude and Codex subscriptions you set me up with, and it's working out quite cost-effectively. In the current setup Codex does the main work of reading the documents and making the sub-asset-class calls, and Claude acts as the independent verifier and settles any conflicting calls. On capability I'm happy with how this is performing; if I ever wanted to trim cost further there are cheaper model options to look at.
- Next step is to finalise the pipeline, fold in your feedback, and run it across all 37 sources. I expect to have those results back to you comfortably within 48 hours, likely within the next day. Once the full run is done I can pull real cost figures from the subscription usage, which should closely track what the equivalent Claude or ChatGPT API access would cost, and later in the week I can start weighing up API pricing against staying on the subscriptions if

you'd like to explore it. This is still the MVP: the priority right now is getting accurate results into your hands, then tidying it into something you can use day to day.

Allocator Pro, proof of concept. Prepared for the Markets Recon team.